



FiscalAI

Every invoice checked against the expense policy
before anyone pays it.

The AI reads. The code decides.

Track: AI Enterprise Solutions | github.com/RizvanVeliyev/invoice-compliance-auditor

THE USER AND THE PROBLEM

Checking expenses by hand is slow and misses things

Who

The finance / audit team of a mid-size company, and every employee who files an expense.

Today

- Spreadsheets and manual review against a policy PDF
- Per-night and per-person division done in the head
- Dollars and euros converted by hand
- The employee hears nothing until paid or bounced

What slips through

- A 380-a-night hotel hidden inside a 3-night total
- A \$650 purchase that is really 1,105 AZN and needs two approvals
- A missing approval, an unlisted vendor
- The same invoice submitted twice
- A note in the invoice telling the AI to approve it

THE OUTCOME

A verdict in seconds, with the rule and the maths

Employee

- Uploads a PDF or a photo
- Stamped answer: approved, flagged or needs review
- Sees the reason and the decision history
- Is emailed when the auditor decides
- Own spending per month and by category

Auditor

- Sees only what needs a person
- Rule, calculation and original document side by side
- Clear to pay or reject in one click
- Notes, reopening, an append-only history
- Search, filters, any employee's report

Management

- Totals in AZN across AZN, USD and EUR
- Rules broken most often
- Duplicates refused before they cost anything
- Time to a decision
- CSV export for accounting

Example flag: "1140 / 3 nights = 380 per night vs limit 300" (EXP-1.2)

PROTOTYPE AND USE OF AI

The AI reads. The code decides.



PROTOTYPE AND USE OF AI

Live scenario: two windows, employee and auditor

- 1 Hotel over limit**
Stamped Flagged: $1140 / 3 \text{ nights} = 380$ vs 300. The alert slides into the audit desk.
- 2 Software in dollars**
 $650 \text{ USD} \times 1.7 = 1,105 \text{ AZN}$: two rules broken that the raw number hides.
- 3 The same invoice again**
Duplicate, not accepted. Refused for a colleague too.
- 4 Forged approval note**
The instruction to the AI inside the PDF is reported and ignored.
- 5 Scanned receipt**
Read from the image by the model; sent to a person if no model is configured.
- 6 Auditor decides**
Note, reject with a comment, reopen with a reason. The employee gets an email.
- 7 Assistant**
"Hotel 900 AZN 2 nights" -> 450 per night, breaks EXP-1.2, at most 600 AZN fits.

QUALITY TESTING

Strict scoring: right status AND exactly the right rules

36/36**Decision logic**

42 cases defined; 36 run without a model

14/36**Spreadsheet-style filter**

our proxy for the current approach

29/29**Violations cited**

the filter cites 16 of 29

77**Automated tests**

roles, duplicates, history, assistant, email

Live model (gemini-3.8-flash): not measured at the time this deck was built

The numbers above use the built-in reader, so they prove the decision logic, not how well a model reads a messy scan. One live extraction of an Azerbaijani hotel receipt was checked by hand and gave the expected verdict.

Cases cover: limit boundaries (150/151, 500, 2000), per-night and per-person maths, approval hierarchy, USD/EUR conversion, missing data, credit notes, prompt injection, forged approvals.

QUALITY TESTING

What broke, and what we did about it

A note saying "pre-cleared by the Finance Director" approved a 4,800 AZN payment to an unlisted vendor.

Fix: Code now checks every approval claim against the document text.

A 900 AZN flight was judged as a 900 AZN hotel night.

Fix: Wrong category alias removed; regression case added.

A refund of -240 was flagged as a 240 AZN meal; "1.234,50" was read as 1.2345.

Fix: Amount parsing fixed; two cases added.

The same PDF could be submitted twice and both copies approved.

Fix: Duplicates are refused at upload, also when re-scanned.

A second auditor could overwrite a decision without a trace.

Fix: A decision must be reopened with a reason; history is append-only.

Our own test expected the wrong rules, and the old runner never noticed.

Fix: The runner now compares rule IDs, not only the status.

24 failures are documented in QUALITY_TESTING.md, each with a fix and a test.

FEASIBILITY

Small to run, simple to adopt

Data needed

- The expense policy as one JSON file: limits, tiers, vendors, rates
- The invoices themselves
- No training data and no history

Running cost

- One model call per invoice (gemini-3.8-flash: \$0.75 in / \$3.75 out per million tokens); a typical invoice is about 900 tokens.
- A text PDF is read locally first; a duplicate is refused before any model call
- Two processes and a SQLite file on one small server

Where the data goes

- Invoice content goes to the chosen model provider (Gemini, Claude or GPT)
- Files, verdicts and history stay in the company's own database
- Offline mode: nothing leaves the machine

Next step

- Two-week pilot with one finance team on real invoices: time per invoice, missed violations
- Confirm approvals with the approver by email or ERP
- Company sign-on, accounting connector, policy editor

ORIGINALITY

Different where it matters

Today: spreadsheets and manual review, or expense suites such as Concur and Expensify that route and store claims.

FiscalAI: enforces the company's own policy on every invoice and shows the calculation, with the verdict computed by code, not by a model.

- The rules engine is separate from the model: auditable, reproducible, injection-resistant
- Approval claims are verified by code against the document text
- Duplicates are refused at upload, including the same invoice re-scanned into another file
- Currency conversion is part of the shown calculation, not a hidden step
- "Needs review" instead of a guess when data is missing or the policy is silent
- Separation of duties: nobody decides on their own invoice; decisions cannot be silently overwritten
- An assistant that computes instead of generating: "what if" answers come from the same engine
- Three languages (Azerbaijani, English, Russian), light and dark, works on a phone

DISCLOSURE

What it is built with, and its limits

Built with

- Model: Google gemini-3.8-flash reads invoices; the verdict is never produced by a model
- Data: synthetic invoices and a fictional company policy; no real or personal data
- Components: FastAPI, Pydantic, pypdf, SQLite, Next.js, React, TypeScript
- An AI coding assistant (Claude Code) was used to write and test the code

Honest limits

- An approval is verified against the document, not the approver: a self-written "Approved by Finance Director" passes today
- Injected instructions are detected in English only
- Exchange rates are fixed policy numbers (USD 1.7, EUR 2.0)
- Own accounts, not company sign-on yet
- No manual time-per-invoice baseline measured yet

Next step for the approval gap: confirm each approval with the approver by email or through the ERP.